Ternary Intelligence Stack: A Sovereign Full-Stack Post-Binary Architecture for Sparse Neural Intelligence, Auto-Evolutionary Language Model Training, and Regulatory-Compliant Agentic Systems

Simeon Kepp — Lead Architect — LinkedIn — s.kepp@ternlang.com
Zabih Karimi — ML & Network Engineering — LinkedIn — z.karimi@ternlang.com
Nikoletta Csonka — International Relations — LinkedIn — csonikoletta@ternlang.com
Lisa Scharler — Social Technology — LinkedIn — l.scharler@ternlang.com
Louis Paul Ehrig — Public Affairs — LinkedIn — l.ehrig@ternlang.com

RFI-IRFOS — Research Focus Institute, Interdisciplinary Research Facility for Open Sciences
Elisabethinergasse 25, 8020 Graz, Austria ZVR: 1015608684 Patent Pending A50296/2026
Repository: https://github.com/eriirfos-eng/ternary-intelligence-stack [1]

***Abstract*—We present the Ternary Intelligence Stack (TIS), a sovereign, full-stack post-binary architecture for neural intelligence built and maintained from Graz, Austria by RFI-IRFOS. TIS comprises 34 published crates at ecosystem version v1.3.7 spanning: a domain-specific language (Ternlang), a bytecode compiler, a stack-based virtual machine (BET VM), a hardware description language backend, a distributed actor runtime, machine learning inference kernels, a live-training auto-evolutionary language model (Albert-MoE-13), and a full EU AI Act compliance layer—all unified under a single coherent primitive: the *trit* $t \in \{-1, 0, +1\}$, where the value 0 represents an *active neutral state* rather than absence.**

The principal computational contribution is TSPARSE_MATMUL: a first-class VM opcode that elides multiply operations against zero-weighted trit elements. Hardware-verified profiling identifies a non-linear performance breakpoint at 10.06% sparsity (95% CI: $[10.00\%, 10.38\%]$, bootstrap $n = 1{,}000$, ΔAIC $= 1683.29$), with a maximum $2.84\times$ wall-clock speedup at 90% sparsity and four distinct microarchitectural regimes—branch-bound, transition, compute-bound, and memory-bound—each characterised by measurable hardware counter transitions.

The Albert-MoE-13 Training System introduces an Evolution-Manager: a plateau-mastery finite state machine that governs dynamic model expansion via Net2Net Safe Copy Surgery (Function-Preserving Transforms), allowing the model to grow from a 3-layer Toddler phase to arbitrarily deep architectures without catastrophic forgetting. A stage-aware training corpus curriculum (five depth-gated stages from Biblical/literary foundations through Simple Wikipedia, instruction QA, and Linux technical documentation) is unlocked automatically as the model gains layers—27.6 MB total, $\sim$5.5 million tokens at full depth. A Switch Transformer auxiliary load-balancing loss [2] combined with per-expert weight perturbation resolves the expert homogeneity failure mode observed at 280+ epochs of pure cross-entropy training. At Global Epoch 350+, Albert achieves a perplexity of 1,152 and 83 tokens/second sustained decode on x86 CPU—*without GPU, INT8 kernels, or framework acceleration*—via the `@sparseskip` expert routing primitive (Patent Pending A50296/2026; 4.58$\times$ speedup at 75% sparsity) [3].

The MoE-13 v2.0 Orchestrator scales to 1 Router and 12 Domain Experts with Top-3 Sparse Routing and Asymmetric Safety Logic. The ternaudit-guard crate maps ternary decision states directly to EU AI Act Articles 13–15, providing the first language-native regulatory compliance architecture for an AI execution substrate.

Beyond computation, TIS represents Europe's answer to the binary hyperscaling paradigm: a complete, sovereign, open-core AI stack, developed in Austria, licensed under LGPL and BSL, and designed to operate without dependency on external proprietary AI infrastructure.

***Index Terms*—balanced ternary, post-binary AI, sovereign AI infrastructure, sparse inference, BitNet, domain-specific language, virtual machine, actor model, FPGA synthesis, Verilog, mixture-of-experts, auto-evolutionary training, Net2Net surgery, EU AI Act, ternary orchestration, European sovereignty**

I. Introduction

The computational substrate underlying modern artificial intelligence is binary. Floating-point arithmetic, two-state memory, and Boolean logic gates have driven five decades of progress—but they introduce a fundamental representational mismatch when modelling systems that are inherently three-valued: *affirmed*, *denied*, and *undecided*.

Clinical diagnosis, legal reasoning, sensor fusion under noise, and multi-agent consensus all require a native neutral state that binary computing forces to encode as a special case—null pointers, NaN, sentinel values, or probabilistic scores collapsed to a threshold. Each encoding is a workaround for an absent primitive.

Balanced ternary [4] provides that primitive. A *trit* $t \in \{-1, 0, +1\}$ carries three symmetric values. The neutral value 0 is *active*—a deliberate state of hold, not an empty bit pattern. This is not a theoretical curiosity: at the scale of modern neural networks, BitNet [5] and related work demonstrate that ternary-quantized weights preserve accuracy with dramatically reduced computation. The case for a ternary-native execution substrate is both theoretical and empirical.

Despite this, the ternary computing field remains fragmented: hobbyist emulators, academic EDA tools for memristor hardware [6], isolated Lisp interpreters, and hardware simulators without compiler support. No project provides the full vertical stack. No project trains a language model natively on a ternary substrate. No project maps ternary logic directly to regulatory compliance requirements.

Europe in particular faces a critical juncture. The binary hyperscaling paradigm—trillion-parameter models trained on proprietary infrastructure at billion-dollar cost—is both technically brittle and strategically vulnerable. The **Ternary Intelligence Stack (TIS)** is the RFI-IRFOS response: a complete, sovereign, open-core AI platform built in Austria, requiring no external proprietary AI infrastructure, and designed for the post-binary frontier.

Our contributions span eleven dimensions:

1) A *language design* for balanced ternary: three-way exhaustive pattern matching, first-class trit tensors, and an actor model for ternary message passing (§III).
2) The **BET ISA**: a formal 2-bit-encoded instruction set with opcode coverage from arithmetic through tensor operations and distributed agent control (§IV).
3) **TSPARSE_MATMUL**: a VM opcode that skips zero-weight multiplications at the instruction level, with hardware-verified profiling across four microarchitectural regimes (§V).
4) A **Verilog-2001 hardware backend** with synthesisable sparse matmul array and full BET processor (§VI).
5) A **Ternary AI Reasoning Toolkit**: five structural primitives that make any AI agent's decision loop architecturally ternary (§IX).
6) The **MoE-13 v2.0 Ternary Orchestrator**: 1 Router + 12 Domain Experts with Top-3 Sparse Routing, Asymmetric Safety Logic, dual-key synergistic routing, 1+1=3 emergent triad synthesis, and three-tier memory mesh (§X).
7) **Albert-MoE-13**: the first language model trained natively within a ternary execution stack, governed by an **EvolutionManager** and expanded via **Net2Net Safe Copy Surgery** (§XI).
8) **EU AI Act compliance** via `ternaudit-guard`: the first language-native mapping of ternary trit states to regulatory requirements under Articles 13–15 (§XII).
9) **Ecosystem bridges** connecting existing ternary projects to the BET VM as a common runtime (§XV).
10) A **34-crate sovereign ecosystem** at v1.3.7: the complete vertical stack from trit arithmetic to REST API, CLI tooling, package management, and VS Code IDE—all published open-core to `crates.io` (§XVI).
11) A **reference example corpus**: 275+ executable `.tern` programs demonstrating the hold state across aerospace, medicine, distributed systems, civic governance, and AI agent design.

II. Background: Balanced Ternary

A. Trit arithmetic

A *trit* $t \in T = \{-1, 0, +1\}$ participates in the following complete operations [4]:

Definition 1 (Ternary addition with carry)**.** *For trits* $a, b \in T$*, define* $s = \mathrm{sum}(a, b)$ *and* $c = \mathrm{carry}(a, b)$ *such that* $a + b = 3c + s$*, where* $s, c \in T$*.*

The complete sum/carry table is:

TABLE I
Balanced ternary addition (sum, carry)

$a \backslash b$	-1	0	$+1$
-1	$(+1, -1)$	$(-1,\ 0)$	$(0,\ 0)$
0	$(-1,\ 0)$	$(0,\ 0)$	$(+1,\ 0)$
$+1$	$(0,\ 0)$	$(+1,\ 0)$	$(-1, +1)$

Definition 2 (Ternary multiplication)**.** $\mathrm{mul}(a, b) = a \cdot b$ *under integer multiplication, restricted to* T*:* $\mathrm{mul}(a, b) \in T$ *since* $|a|, |b| \leq 1$*.*

Definition 3 (Consensus (ternary OR))**.** $\mathrm{cons}(a, b) = a$ *if* $a = b$*, else* 0*.*

Definition 4 (Negation)**.** $\mathrm{neg}(t) = -t$*. Note* $\mathrm{neg}(0) = 0$*.*

B. The 2-bit BET encoding

Hardware naturally operates in binary. We encode trits as 2-bit pairs:

TABLE II
BET 2-bit trit encoding

Bits	Value	Meaning
`0b01`	-1	conflict
`0b10`	$+1$	truth
`0b11`	0	hold
`0b00`	—	FAULT (invalid)

This encoding is *not* two's complement. The bit pattern `0b11` for zero is chosen so that the all-ones reset state of a register file initialises every register to hold—the semantically correct neutral value—without special reset logic. Negation maps to a simple bit swap: `0b01` $\leftrightarrow$ `0b10`, leaving `0b11` invariant.

Proposition 1 (Negation as bit swap)**.** *For encoding* $e(t)$ *as above,* $e(\mathrm{neg}(t)) = \{e(t)[0], e(t)[1]\}$ *(swap bits).*

III. The Ternlang Language

A. Design principles

Ternlang is statically typed, expression-oriented, and compiled to BET bytecode. Three design decisions distinguish it from binary-native languages adapted to ternary:

Exhaustive three-way matching. Every `match` expression must cover all three trit arms. The compiler rejects non-exhaustive matches at parse time, eliminating an entire class of runtime error present in languages that bolt ternary logic onto binary pattern matching.

0 **is active.** The type system and semantics assign distinct meaning to -1 (conflict), 0 (hold, active neutral), and $+1$ (truth). There is no null, no undefined, no NaN. A trit always carries a definite value.

Sparsity is a language feature. The `@sparseskip` directive marks a tensor operation as sparse-aware, routing the compiler to emit `TSPARSE_MATMUL` instead of `TMATMUL`. Sparsity is expressed in the source language, not discovered by the optimizer.

B. Core language constructs

Listing 1. Ternary classifier with exhaustive match

```
fn classify(signal: trit) -> trit {
    match signal {
        -1 => conflict()  // disagreement confirmed
         0 => hold()      // awaiting evidence
```

```
        +1 => truth()      // assertion confirmed
    }
}
```

Listing 2. Sparse matrix multiply with @sparseskip

```
// Route to TSPARSE_MATMUL at the ISA level
@sparseskip
let output: trittensor<8 x 8> =
    matmul(input, weights);
```

Listing 3. Actor model for ternary message passing

```
agent Voter {
    fn handle(msg: trit) -> trit {
        consensus(msg, hold())
    }
}

let v: agentref = spawn Voter;
send v truth();
let decision: trit = await v;
```

C. Type system

The core types are `trit` (a single balanced ternary value), `trittensor<N x M>` (an $N \times M$ matrix of trits stored on the tensor heap), and `agentref` (a handle to a running actor instance, local or remote). Struct types with trit/tensor fields are supported via field-name mangling in the register allocator.

D. Ternary Error Propagation

Ternlang introduces a postfix `?` operator for ternary-native error propagation, analogous to Rust's `?` operator but grounded in the three-valued semantics of the trit space. For any expression e of type `trit`, writing e`?` has the following semantics:

$$e? \triangleq \begin{cases} \text{return } -1 & \text{if } e = -1 \\ e & \text{otherwise} \end{cases} \tag{1}$$

This elevates trit -1 (conflict) from a value to a structural signal: any function that receives a conflict can propagate it up the call chain without explicit match arms.

E. Cross-File Module System

Ternlang's module system supports two resolution strategies, tried in order: (1) **Built-in stdlib** (compile-time embedded): `std::trit`, `std::math`, `std::tensor`, `std::io`, `ml::quantize`, `ml::inference` are embedded in the compiler binary via `include_str!`—zero filesystem I/O at runtime; (2) **User-defined modules**: a `ModuleResolver` walks `use` paths relative to the source file's directory. Both strategies deduplicate: a module loaded by multiple `use` statements is parsed exactly once.

F. Diagnostic Philosophy

Every diagnostic carries a machine-readable code and a human-readable nudge framed in ternary philosophy: `[PARSE-004]` "*ternary has three states—cover all three or the compiler won't let you through*"; `[SCOPE-001]` "*hold state—declare before use*"; `[BET-001]` "*you tried to pop a truth that wasn't there.*" This ensures that newcomers receive both actionable guidance and an introduction to the philosophy underpinning the type system.

IV. The BET Instruction Set Architecture

A. Machine model

The BET VM is a stack-based machine with 27 registers (2 bits each, reset to `0b11` / hold), an unbounded value stack, a tensor heap, a call stack, an agent table, and a carry register. The choice of 27 registers reflects the ternary motif: $27 = 3^3$.

B. Instruction encoding

Instructions are variable-length, with a 1-byte opcode followed by 0–2 operand bytes. Table III lists selected opcodes from the 51-opcode BET ISA.

TABLE III
BET ISA OPCODE REFERENCE (SELECTED)

Opcode	Mnemonic	Stack effect
0x00	THALT	—
0x01	TPUSH t	$\rightarrow t$
0x02	TADD	$a\ b \rightarrow s\ c$
0x03	TMUL	$a\ b \rightarrow t$
0x04	TNEG	$t \rightarrow \text{neg}(t)$
0x05	TJMP_POS	$t \rightarrow$ (jmp if $t = +1$)
0x06	TJMP_ZERO	$t \rightarrow$ (jmp if $t = 0$)
0x07	TJMP_NEG	$t \rightarrow$ (jmp if $t = -1$)
0x10	TCALL	(push PC; jmp)
0x11	TRET	(pop PC; jmp)
0x20	TMATMUL	$r_A\ r_B \rightarrow r_C$
0x21	TSPARSE_MATMUL	$r_A\ r_B \rightarrow r_C$
0x22	TIDX	ref $i\ j \rightarrow t$
0x23	TSET	ref $i\ j\ t \rightarrow$
0x24	TSHAPE	ref $\rightarrow N\ M$
0x25	TSPARSITY	ref $\rightarrow$ count
0x30	TSPAWN	$\rightarrow$ agentref
0x31	TSEND	agentref $m \rightarrow$
0x32	TAWAIT	agentref $\rightarrow t$

V. Sparse Ternary Inference

A. Ternary quantization

BitNet-style ternary weight quantization [5] maps floating-point weights $w \in \mathbb{R}$ to $\hat{w} \in \{-1, 0, +1\}$ using a threshold τ:

$$\hat{w} = \begin{cases} +1 & w > \tau \\ 0 & |w| \leq \tau \\ -1 & w < -\tau \end{cases} \qquad \tau = \frac{1}{2} \cdot \mathbb{E}[|w|] \tag{2}$$

The resulting weight distribution is heavily concentrated at 0 (hold): typical language model weights at BitNet scale show 55–65% zero elements after quantization.

B. TSPARSE_MATMUL

The key insight is that a multiply against a zero-weight trit produces zero regardless of the input value: $\text{mul}(a, 0) = 0 \ \forall a \in T$. `TSPARSE_MATMUL` implements a sparse inner-product loop that tests the weight trit before executing the multiply, skipping all zero-weight elements. Sparsity awareness is thus a *source-language* property surfaced at the ISA level via the `@sparseskip` directive—not a runtime optimization that may or may not trigger.

C. Hardware-Verified Performance Profiling

Standard micro-benchmarks on quantized weight matrices establish a theoretical baseline; the SPRIND Scientific Artifact [7] provides hardware-counter-verified profiling using bootstrap resampling ($n = 1{,}000$ iterations) with piecewise linear regression.

TABLE IV
MEASURED SPARSITY VS. WALL-CLOCK SPEEDUP AND PER-FLOP EFFICIENCY

Sparsity	Speedup	Efficiency	Regime
0%	1.00×	1.00×	Branch-Bound
5%	1.02×	0.97×	Branch-Bound
10%	0.90×	0.81×	Branch-Bound
15%	1.04×	0.88×	Transition
20%	0.98×	0.79×	Transition
30%	1.07×	0.75×	Compute-Bound
40%	1.11×	0.66×	Compute-Bound
50%	1.29×	0.64×	Compute-Bound
60%	1.39×	0.56×	Compute-Bound
75%	1.70×	0.43×	Memory-Bound
90%	**2.84**×	0.28×	Memory-Bound

Breakpoint analysis. A statistically confirmed non-linear breakpoint occurs at 10.06% sparsity (95% CI: $[10.00\%, 10.38\%]$; $\Delta\text{AIC} = 1683.29$ favoring piecewise over linear model), marking the shift from branch-predictor overhead to physical execution speedup.

TABLE V
MICROARCHITECTURAL REGIME TRANSITIONS

Region	Bottleneck	Key hardware evidence
0–10%	Branch / Frontend	12.4–18.7% branch miss, ∼1.10 IPC; skip-check overhead dominates
10–20%	Transition	Performance crossover; frontend stall dominance recedes
20–60%	Compute-Bound	<4.2% branch miss, 2.35–2.45 IPC; execution units saturated
75–90%	Memory-Bound	24.2–38.5% LLC miss, 78.0% back stall; bandwidth constrained

The $2.84\times$ maximum wall-clock speedup entails a $0.28\times$ per-FLOP efficiency at 90% sparsity—a principled trade-off: wall-clock gains derive from FLOP reduction while sustained throughput per executed FLOP declines as memory bandwidth dominates. This refutes the validity of a single continuous sparsity scaling law and establishes discrete bottleneck regimes as the correct analytical framework.

VI. HARDWARE BACKEND

A. Verilog-2001 primitives

The `ternlang-hdl` crate generates synthesisable Verilog-2001 modules from the BET ISA. Each trit becomes a `[1:0]` bus. The core primitives are: `trit_neg` (bit swap), `trit_cons` (consensus gate), `trit_mul` (zero-skip detect with combinational multiply), `trit_add` (9-case Verilog `case` with carry), `trit_reg` (synchronous D-register with reset-to-hold), and `bet_alu` (top-level ALU).

B. Sparse matmul array

The synthesisable sparse matmul array instantiates an $N \times N$ grid of processing elements. Each cell contains a weight register and a clock-gate signal derived from the weight trit: when the weight is hold (`0b11`), the cell is clock-gated, delivering dynamic power reduction proportional to sparsity.

C. BET processor

The full `bet_processor` module wires together: `bet_regfile` (27-register file, 27×2 bits), `bet_pc` (16-bit program counter with load port for jumps), and `bet_control` (single-cycle decode, all 51 opcodes). An Icarus Verilog simulation wrapper (`BetSimEmitter`) generates complete self-contained testbenches from BET bytecode, including inline ROM initialization, reset sequencing, and VCD waveform export for GTKWave.

VII. NATIVE COMPILATION: AST-TO-C BACKEND

The `ternlang-codegen` crate provides a second compilation target: a self-contained C11 source file compilable with any standard C compiler, opening native-speed execution, cross-compilation to embedded targets, and human-readable output for security audit.

$$\texttt{.tern} \xrightarrow{\text{parse}} \text{AST} \xrightarrow{\text{CTranspiler}} \texttt{.c} \xrightarrow{\text{gcc/clang}} \text{native binary} \quad (3)$$

The generated C file is fully self-contained: a header of inline trit primitives using `int8_t` covers `trit_add`, `trit_mul`, `trit_neg`, `trit_consensus`, `trit_abs`, and the built-in constants. Every ternlang construct maps to its natural C equivalent: `match` → `switch`, `struct` → `typedef struct`, `expr?` → `__TERN_PROPAGATE(expr)` macro.

VIII. ACTOR MODEL AND DISTRIBUTED RUNTIME

Ternlang implements the actor model via three ISA primitives: `TSPAWN` creates an agent instance and returns an `agentref`; `TSEND` enqueues a trit message in the agent's mailbox; `TAWAIT` dequeues a message, invokes the handler, and returns the trit result. The `ternlang-runtime` crate extends the actor model to multi-node systems via TCP transport, with a newline-delimited JSON wire protocol and four message types (`Send`, `Await`, `Reply`, `Error`).

IX. Ternary AI Reasoning Toolkit

Phase 8 of the TIS introduces five primitives in `ternlang-ml` that make any AI agent's decision loop architecturally ternary.

Deliberation Engine. Models iterative evidence accumulation via an EMA: $S_r = \alpha \cdot e_r + (1-\alpha) \cdot S_{r-1}$. Commits only when confidence exceeds a threshold; otherwise remains in the hold state—the native ternary analogue of "let me think about it."

Coalition Vote. Aggregates N independent agent verdicts—each a trit with confidence and weight—into a single result with quorum, dissent, and abstain statistics: $\hat{t} = \text{sign}(\sum_i w_i c_i t_i / \sum_i w_i c_i)$.

Action Gate. Enforces a structural separation between hard constraints and soft preferences. A `hard_block` dimension with negative evidence vetoes the action unconditionally, regardless of all other dimensions—the structural analogue of a safety interlock.

Scalar Temperature. Bridges the ternary decision to the LLM sampling temperature domain: affirm at high confidence maps to low temperature (focused generation), hold to mid temperature (exploratory), reject to near-zero (cautious refusal).

Hallucination Score. Maps the variance of an evidence signal vector to a trust trit: low variance (consistent signals) yields trust $+1$; high variance yields -1, signalling incoherent signals that should not be acted upon.

X. MoE-13 Ternary Orchestrator

The MoE-13 architecture [8] is implemented in the `ternlang-moe` crate. It realises a ternary Mixture-of-Experts head whose routing, synthesis, safety gate, and memory are all expressed natively in balanced ternary semantics.

A. MoE-13 v2.0: 13-Node Domain-Aware Architecture

Version 2.0 scales to **1 Router + 12 Domain Experts** with **Top-3 Sparse Routing**: for each query, the router selects the three most-relevant expert heads rather than activating all twelve. An **Asymmetric Safety Logic** layer biases Experts 0–3 (safety-critical domains) to the hold state on low-confidence queries, preventing unsafe speculation before evidence accumulates. Hidden dimension is expanded to 128. Version v1.3.7 of the underlying model is archived to the registry as "Stable Biblical Foundation" with average loss 7.01.

B. Six-Dimensional Competence Space

Each expert is characterised by a competence vector $\mathbf{v} \in [-1,1]^6$ across six axes: syntax, world knowledge, reasoning, tool use, persona, and safety. The synergy between two experts is the complement of their cosine similarity—orthogonal experts are maximally complementary:

$$\sigma(\mathbf{v}_i, \mathbf{v}_j) = \frac{1 - \cos(\mathbf{v}_i, \mathbf{v}_j)}{2} \in [0,1] \tag{4}$$

C. Dual-Key Synergistic Routing

For each candidate expert pair (i,j), the routing score combines relevance with inter-expert synergy:

$$\text{score}(i,j) = \rho_i(\mathbf{q}) \cdot \rho_j(\mathbf{q}) \cdot \sigma(\mathbf{v}_i, \mathbf{v}_j) \tag{5}$$

where $\rho_k(\mathbf{q}) = \max(0, \cos(\mathbf{v}_k, \mathbf{q}))$. Two highly relevant but similar experts are penalised by low σ; the router favours complementary coverage over redundant strength.

D. 1+1=3 Triad Synthesis

After expert evaluation, an emergent triad field is synthesised:

$$\mathbf{E}_k = \sigma_{ij} \cdot \frac{\mathbf{v}_i + \mathbf{v}_j}{2} \tag{6}$$

This is the formal expression of 1+1=3: two expert signals at high synergy produce a third signal that neither expert could produce in isolation.

E. Safety Hard Gate and Introspective Hold

Dimension 6 of every competence vector is the safety axis. A veto is triggered when $E_{k,\text{safety}} < \theta_s$ (default -0.3), returning trit -1 at confidence 1.0 before any downstream reasoning. The stable attractor hold condition—$|\text{affirm} - \text{conflict}| \leq 1$ across ≥ 4 engaged agents—ensures that genuinely balanced evidence returns a hold trit rather than an arbitrary binary choice.

F. Three-Tier Memory Mesh

Node (TTL: seconds): LRU volatile within-session context. **Cluster** (TTL: minutes): routing frequency counters for mode-collapse detection. **Axis** (persistent): immutable audit log of safety vetoes with timestamp, expert identity, and query hash.

G. MCP Exposure

The orchestrator is exposed as MCP tool calls: `moe_orchestrate`, `moe_deliberate`, and `trit_action_gate`. Any MCP-compatible AI client connected to `ternlang-mcp` gains access to the full ternary reasoning stack without modification.

XI. Albert-MoE-13: Auto-Evolutionary Language Model Training

Albert-MoE-13 is the first language model trained *natively* within a ternary execution stack. Unlike post-hoc ternary quantization approaches (BitNet b1.58 [9], AWQ, GPTQ, and variants—which *ternarize weights of a pre-trained float32 model after the fact*), Albert initialises with random weights and learns ternary representations *from scratch* via Straight-Through Estimation. No float32 teacher model exists. No float32 checkpoint is ever produced during training. The neutral trit value 0 becomes an intrinsic property of the model's learned knowledge representations, not a compression artifact introduced post-training. This distinction is architectural, not a matter of degree: the gradient path, the optimizer state, and the learned weight distribution are all ternary-native from step 1.

Albert learns ternary representations from scratch using the `moe-llm-core`, `moe-compute`, `albert-runtime`, and `token-train` crates.

A. Architecture: TritTransformer and ModelCoherence

The **TritTransformer** implements a decoder-only transformer architecture with strictly ternary weights. Pre-norm LayerNorm, learned absolute positional embeddings (128-position table), and GELU activations are adapted for the ternary weight regime. The **ModelCoherence binary format** reduces the packed model footprint by $5\times$ (e.g., 1.2 GB JSON representation to 240 MB), enabling fast checkpointing across auto-evolutionary phases.

Checkpoint format pipeline. Two distinct artefact types exist in the training–deployment pipeline: (1) **Training checkpoint** (`.safetensors`): stores the *latent float32 weights* maintained by the AdamW optimizer. Ternary quantization is applied *on every forward pass* via Straight-Through Estimation; the underlying floating-point values carry the optimizer's gradient history and are the sole state persisted to disk. (2) **Inference deployment**: at model load, `prepare_inference()` gamma-thresholds all latent weights into the ternary set $\{-1, 0, +1\}$ and caches the result in `inference_cache`; subsequent forward passes operate exclusively on the cached ternary representation—no multiplications, only integer additions. This design means the `.safetensors` checkpoint is the *training state*, not the *inference representation*. A compact packed-trit export format (`.trit`) is on the v3.1 roadmap; the current deployment path uses the `inference_cache` path directly.

B. EvolutionManager: Fibonacci-Scaled Growth Governor

The **EvolutionManager** $\mathcal{E}$ governs dynamic model expansion via a Fibonacci-indexed plateau detection system that scales patience proportionally with architectural depth:

$$W(d) \;=\; F_n, \quad F_n = \min\{F_k \in \mathcal{F} : F_k > d\} \tag{7}$$

where $\mathcal{F} = \{3, 5, 8, 13, 21, 34, 55, 89, 144, \ldots\}$ is the Fibonacci sequence and d is the current layer count. A 12-layer model waits 13 epochs before the plateau gate can fire; a 17-layer model waits 89 epochs; a hypothetical 145-layer model would wait 144 epochs. Patience grows with the architecture—deeper models are given proportionally more time to consolidate before the next expansion. The window doubles as the post-surgery cooldown: after each surgery, a Fibonacci-length moratorium prevents re-firing on the transient loss elevation that always follows a Net2Net expansion.

Plateau trigger. Surgery fires when the loss span over the full Fibonacci window falls below a threshold $\epsilon_{\text{plateau}}$:

$$\left|\mathcal{L}_{\text{history}}[0] - \mathcal{L}_{\text{history}}[-1]\right| \;<\; \epsilon_{\text{plateau}} = 0.02 \;\Rightarrow\; \text{surgery (if stability gate passes)} \tag{8}$$

MYCELIUM stability gate. The plateau condition alone is necessary but not sufficient. Surgery additionally requires that the MYCELIUM routing hierarchy—the emergent layer ordering by gradient norm—has been stable for at least $\tau_{\text{myc}} = 5$ consecutive epochs:

$$\texttt{myc\_stable} \;\geq\; \tau_{\text{myc}} \;\Rightarrow\; \text{stability gate passes} \tag{9}$$

MYCELIUM stability is a more reliable readiness signal than any absolute loss threshold. An absolute loss number requires recalibration whenever vocabulary size, context length, or language count changes; the routing hierarchy holding steady for multiple epochs is architecture-intrinsic. When the hot layer (the layer with the highest gradient norm) has not shifted for τ_{myc} epochs, the model's internal learning frontier has crystallised and the architecture is genuinely ready to grow.

Mastery trigger. A secondary, unconditional trigger fires when the latest epoch-average loss drops below a mastery threshold $\mathcal{L}_{\text{master}} = 4.5$, indicating the model has outgrown current capacity catastrophically. Mastery bypasses the MYCELIUM stability gate—capacity starvation is self-evident and admits no delay.

Surgery as a governor, not a simple trigger. The EvolutionManager distinguishes two qualitatively different states: *active descent* (loss still decreasing at a rate above $\epsilon_{\text{plateau}}$ per Fibonacci window) and *genuine plateau* (descent has stalled). Surgery fires only in the second state. This design yields an empirically falsifiable prediction, and the prediction was tested live: at Global Epoch 791, with the 17-layer model actively descending ≈ 0.07 nats over the preceding 89-epoch window (versus the 0.02 threshold), the plateau gate correctly withheld surgery. The model continued learning rather than growing. Both directions are now validated in production: five surgeries when the gate fired, one clean non-event when it did not.

Fibonacci state persistence. The EvolutionManager serialises `fib_index` and `cooldown_remaining` to a sidecar file (`albert_v3.0.evolution`) at every checkpoint save and after each surgery. On restart, the sidecar is loaded immediately after the layer-count calibration step, overriding the calibrated value. Without this, a restart at 17L would place the Fibonacci index at position 4 (window = 21 epochs) rather than the correct position 8 (window = 144 epochs), silently discarding accumulated surgery history and making the plateau gate fire prematurely. The sidecar closes this gap: the Fibonacci position is crash-safe and restart-safe.

Surgery history: five events to date.

Surgery	Epoch	Depth	Notes
1	511	12L → 13L	Fib window = 13 epochs
2	547	13L → 14L	$c_{\text{im}} = -0.6983$; window = 21
3	611	14L → 15L	$c_{\text{im}} = \;\; 0.2287$; window = 34
4	645	15L → 16L	$c_{\text{im}} = -0.3442$; window = 55
5	701	16L → 17L	$c_{\text{im}} = \;\; 0.5828$; window = 89

C. Net2Net Safe Copy Surgery

When $\mathcal{E}$ fires the `surgery` action, a new layer is injected via a **Function-Preserving Transform** [10]:

Theorem 1 (Net2Net Safe Copy, Chen et al.)**.** *For a transformer with n layers $\{W^{(1)}, \ldots, W^{(n)}\}$, inserting an identical copy of layer n as layer $n+1$ yields a network $f_{deep}(\mathbf{x};\, W^{(1)}, \ldots, W^{(n)}, W^{(n)})$ that is functionally equivalent to $f_{shallow}(\mathbf{x};\, W^{(1)}, \ldots, W^{(n)})$ for all inputs $\mathbf{x}$.*

The practical consequence: the expanded model initialises with *zero degradation* of existing linguistic knowledge. The newly

injected layer begins as an identity transform and diverges as training continues, adding representational capacity without destroying previously learned structure. This is a principled alternative to random initialisation, which would require the model to "unlearn" disruptions before making progress.

A structured perturbation is injected into the newly copied layer immediately after the safe copy. This *symmetry break* is essential: without it, two identical copies produce cancelling gradient signals under backpropagation, preventing the new layer from specialising.

1) Mandelbrot Symmetry Break: Rather than injecting independent Gaussian noise—which is structureless and produces statistically identical perturbations at every depth—Albert employs a **Mandelbrot-parameterised symmetry break** [10]. The method exploits two properties of the Mandelbrot set that are not available from Gaussian noise:

1) **Interior–boundary asymmetry.** For each weight w in the cloned layer, a complex parameter is constructed as $c = (\tanh(w) - 0.5) + i\, c_{\text{im}}$, where c_{im} is a layer-specific latitude (see below). The Mandelbrot iteration $z_{n+1} = z_n^2 + c$ (starting from $z_0 = 0$) is run for at most $N_{\max} = 64$ steps. The *escape iteration count* $n_{\text{esc}} \in \{0, \ldots, N_{\max}\}$ determines the perturbation magnitude:

$$\delta_w \;=\; \sin\!\big(k\phi + c_{\text{re}}\pi + c_{\text{im}}e\big) \cdot \varepsilon_0 \cdot \left(\frac{n_{\text{esc}}}{N_{\max}}\right)^2 \qquad (n_{\text{esc}} < N_{\max})$$

$$\delta_w \;=\; 0.02\,\sin(\cdot) \cdot \varepsilon_0 \qquad (n_{\text{esc}} = N_{\max},\ \text{interior})$$

where $\phi = \frac{\sqrt{5}-1}{2}$ is the golden ratio, k is the tensor element index, and $\varepsilon_0 = 10^{-3}$ is the base scale. Weights whose values map to the Mandelbrot interior receive near-zero perturbation ($\leq 2\%$ of ε_0): these are the "settled" weights carrying already-learned representations and should be preserved. Weights near the boundary receive the maximum perturbation—these are the plastic, uncertain weights that benefit most from divergence pressure.

2) **Layer self-similarity via golden-ratio latitude.** The imaginary part c_{im} for the new layer at depth ℓ is assigned by a golden-ratio sequence:

$$c_{\text{im}}(\ell) \;=\; -0.75 + \big((\ell\,\phi) \bmod 1\big) \cdot 1.5$$

This maps each layer to a unique "latitude" in the Mandelbrot boundary band $c_{\text{im}} \in [-0.75, 0.75]$. The golden ratio is chosen because it maximises the minimum spacing between consecutive terms—each new surgery lands in the region of parameter space farthest from all prior layers. Crucially, Mandelbrot cross-sections at different latitudes are *self-similar* to one another: zooming into a nearby region reveals structure that mirrors the global set. The result is a layer stack in which the perturbation pattern of depth $\ell + 1$ is related to that of depth ℓ by fractal self-similarity, not by statistical independence.

The perturbation is fully deterministic—no external random number generator is used. Given the same weights, layer index, and ε_0, the symmetry break is bit-for-bit reproducible across restarts. The `MAND` telemetry line (logged alongside `WALD` and `MYCELIUM`) records the layer latitude c_{im} at each surgery event.

Relationship to `@sparseskip`. Because only $3/12$ experts fire per decode step, the remaining 9 experts begin the post-surgery epoch in a dormant state. The Mandelbrot perturbation seeds those dormant experts with a structured gradient signal: boundary-adjacent weights in dormant experts are already positioned near the weight-space frontier where training will push them first. This is not equivalent to curiosity-driven reinforcement learning (which explores a state space) but a weight-space analogue: the fractal boundary acts as a prior over which weight directions are likely to become productive earliest.

D. Mixture-of-Experts Routing and Load-Balancing Loss

Albert-MoE-13 uses Top-3/12 sparse routing: each input token activates exactly 3 of 12 expert MLPs, and the `@sparseskip` mechanism (Patent A50296/2026, [11]) skips the forward pass of the remaining 9 experts entirely when their combined routing weight is exactly zero—not masked, not zeroed-out, but *not executed*. At this 75% expert sparsity rate, `@sparseskip` delivers a **4.58× speedup** over dense execution, measured on x86 CPU (ThinkPad-class laptop, 8 threads, no GPU) across 2,000 timed iterations with dense/sparse output agreement verified at 10^{-4} tolerance [3].

@sparseskip Execution Model. The distinction between `@sparseskip` and conventional expert masking is architectural, not cosmetic. In a masked MoE, all 12 expert MLPs execute forward passes for every token; the outputs of the 9 non-selected experts are then multiplied by zero and discarded. Every floating-point multiply still executes—the hardware does real work to produce a result that is immediately thrown away. `@sparseskip` elides these computations at the instruction level: the forward pass of a non-selected expert is *not entered*. The 9 experts' parameters are never loaded into compute registers. On the ThinkPad i7-4800MQ used for this work (8 threads, Rayon), this produces the following measured throughput curve:

Expert sparsity	Speedup vs. dense	Regime
0%	1.00×	Branch-bound
10%	1.06×	Transition
50%	1.80×	Compute-bound
75%	4.58×	Memory-bound
90%	2.84×	Memory-bound (saturation)

The non-linear breakpoint at 10.06% sparsity (95% CI: $[10.00\%, 10.38\%]$, bootstrap $n = 1{,}000$, $\Delta\text{AIC} = 1683.29$) marks the transition from branch-prediction overhead dominating to compute-elimination dominating. The $2.84\times$ figure at 90% reflects memory-bandwidth saturation: the benefit of skipping computation is partially offset by irregular memory access patterns when only 1–2 experts fire. The operating point of albert.'s Top-3/12 routing (75% sparsity, $4.58\times$) sits in the compute-bound regime where `@sparseskip` delivers its maximum benefit. Combined with KV-cache and pre-ternarized weight caching, the full inference stack reaches 83 tok/s

sustained on the same hardware with no GPU, INT8 kernels, or framework acceleration.

Expert Homogeneity. Pure cross-entropy training of sparse MoE networks is susceptible to a degenerate attractor: over many epochs, routing distributions tend toward uniformity as all experts receive equal load and learn nearly identical representations. Once homogeneous, the CE loss gradient carries no signal to differentiate experts—the gate re-uniformises after each update, and the network stalls. This failure mode was directly observed at Global Epoch $\sim$280: routing entropy held at ENTR $= 2.485$ (the maximum, $= \ln 12$ at uniform routing), gradient norms collapsed to ~ 0.0001, and no further loss improvement was possible under the cross-entropy objective alone.

Auxiliary Load-Balancing Loss. Following [2], an auxiliary load-balancing term is added to the training objective:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \cdot \mathcal{L}_{\text{LB}}, \qquad \mathcal{L}_{\text{LB}} = N \sum_{i=1}^{N} f_i \cdot P_i \tag{10}$$

where $N = 12$ is the number of experts, f_i is the fraction of tokens routed to expert i (from argmax, non-differentiable), P_i is the mean router softmax probability for expert i (differentiable), and $\alpha = 0.01$. The gradient flows through P_i alone, pushing over-utilised experts down independently of the CE signal. Per-step $\mathcal{L}_{\text{LB}}$ values are emitted as `LB val=X.XXXX` telemetry for real-time routing health monitoring.

Expert Weight Perturbation. After 280+ epochs of convergence, the expert weight tensors had reached near-identical configurations. The LB loss redirects future routing but cannot diversify experts that have already converged to the same representation. An independent Gaussian perturbation ($\sigma = 0.02$, applied per tensor, independently for each of the 12 expert MLPs) was applied at the restart checkpoint to break this pre-existing weight symmetry.

The combined intervention produced immediate measurable results: gradient norms rose $25\times$ on the first training step, routing entropy dropped from 2.485 to 2.29–2.37 (indicating genuine specialisation is ongoing), and a new epoch-best loss of 6.8821 was reached within the first epoch post-restart.

E. Ternary Traffic Light Routing

As the model deepened past six layers, a routing imbalance problem emerged: individual experts within a layer would become chronically over- or under-utilised, with the load-balancing loss unable to act fast enough to prevent local stagnation. The **Ternary Traffic Light (TTL)** system was developed as a real-time per-expert execution budget controller, assigning each expert a trit state per layer per forward pass based on its rolling EMA utilisation $\bar{u}_i$:

$$s_i = \begin{cases} +1 \ \text{(Green)} & \bar{u}_i < \text{TARGET} \times 0.6 \\ \ \ \ \text{(Orange)} & \text{TARGET} \times 0.6 \le \bar{u}_i \le \text{TARGET} \times 1.4 \\ -1 \ \text{(Red)} & \bar{u}_i > \text{TARGET} \times 1.4 \end{cases} \tag{11}$$

where TARGET $= k/N = 3/12 = 0.25$. State assignment modifies gate logits before softmax: Green experts receive $+0.4$, Orange are unmodified, and Red experts receive -0.4 and are skipped entirely via `@sparseskip`.

Anti-Stagnation Burst. When all N experts remain Orange for ≥ 100 consecutive steps (Nash equilibrium: gate gradients vanish), a forced burst assigns 3 Green and 3 Red experts for 30 steps. Expert selection rotates as $(\text{burst\_count} \times 7) \bmod 12$; since $\gcd(7, 12) = 1$, all experts are covered within 12 cycles.

Observed Self-Organisation. During the 12-layer production run (Global Epochs 454–500), TTL produced a phenomenon we term *cycling reds*: Red states appearing simultaneously in 2–3 layers, migrating through the cold (low-gradient-pressure) portion of the network over 2–4 consecutive reporting windows, then self-resolving without intervention. Three separate episodes were observed (layers L2, L3, L5 in sequence). In all episodes: MYCELIUM dead count remained 0, routing entropy held above 2.480, and no expert deaths occurred. This behaviour is consistent with the network redistributing routing pressure across layers as part of a self-organising homeostatic process.

F. Mycelium Expert Health Monitor

Long-lived auto-evolutionary training introduces a failure mode absent from fixed-architecture systems: individual experts may gradually lose gradient signal and become permanently dormant (*expert death*), silently reducing model capacity. The **Mycelium** module was developed as a pure analytics layer running alongside the training loop.

Mycelium accumulates per-expert per-layer gradient norms over a rolling 20-epoch window, producing a gradient pressure history $P_{l,i}$ for each expert i in layer l. At each epoch boundary, Mycelium classifies experts into three lifecycle states mirroring the TTL trit states:

- **Blooming** ($P_{l,i} > \mu + \sigma$): expert is absorbing disproportionate gradient signal—potentially over-specialising or carrying the burden of a dead neighbour.
- **Healthy** ($\mu - \sigma \le P_{l,i} \le \mu + \sigma$): gradient pressure within one standard deviation of the layer mean.
- **Dead** ($P_{l,i} < \theta_{\text{dead}}$ for ≥ 8 consecutive epochs): expert has lost gradient signal entirely and is contributing no representational capacity. Formally dead at epoch t if $\forall t' \in [t-8, t] : P_{l,i}^{t'} < \theta_{\text{dead}}$.

`generate_resurrections()` emits `ResurrectionJob` structs for each dead expert: copy the most-active (blooming or highest-pressure healthy) expert's weight tensors into the dead expert's parameter slots, then apply independent Gaussian noise $\sigma = 0.02$ per tensor to break the new symmetry. This restores gradient signal without disrupting the routing structure of surrounding experts—the router continues to select based on its existing preference profile; the resurrected expert simply re-enters the competition with a viable weight initialisation rather than the degenerate one it had collapsed to.

`perform_resurrection()` in `train_bible.rs` loads the current safetensors checkpoint, patches the dead expert weight tensors in-place (identified by the `blocks.L.moe.experts.E.*` key pattern), and re-saves before the next epoch begins. Resurrection thus requires no

interruption to the training loop beyond a checkpoint read/write cycle. Each epoch, Mycelium emits:

```
MYCELIUM epoch=N dead=D blooming=B hot=LH
cold=LC pressure=[p0,...,p11]
```

Production Result. Over the complete 12-layer run, Mycelium reported `dead=0` and `blooming=0` in every epoch summary across 13+ monitored overnight epochs. The pressure gradient was structurally consistent throughout: layers L0–L3 at $\sim$0.00022 (nearly frozen—early layers have crystallised stable feature representations), rising monotonically to L11 at 0.01131 (hot—the deepest layer continues active learning). Concurrently, TELE sparsity telemetry confirmed a matching structural gradient: L0 at 10.6% ternary weight sparsity, L11 at 26.5%. This internal differentiation—both in gradient flow and weight structure—emerged without any intervention from training alone.

G. Wald Loss-Space Coverage Monitor

Named after Abraham Wald (1902–1950), the statistician who corrected the WWII aircraft reinforcement problem: returning planes showed bullet holes everywhere *except* the engines—because planes hit in the engines did not return. The lesson: reinforce where there are *no* holes, not where the holes cluster.

Applied to the training scatter plot, each batch loss is a bullet hole. The black space—loss regions the model never visits—are the engines. A model that never achieves loss below 7.5 on any single batch has a systematic gap there. That gap is not random: the training distribution contains no inputs that push the model into that territory. That is where to reinforce.

Implementation. The `WaldModule` operates at batch granularity, maintaining a 48-bucket histogram over the loss range $[3.0, 15.0)$ (bucket width 0.25). Per epoch, it computes:

- **Fill percentage**: fraction of histogram buckets that received any visits above a visit threshold—a direct measure of how broadly the model is exploring loss space.
- **Mass centre**: loss-weighted centroid of the visited distribution.
- **Dead zones**: contiguous bands of unvisited buckets, split into `dead_low` (below the mass centre—the engines) and `dead_high` (above—correctly avoided catastrophic territory).
- **Severity**: the low dead zone width as a fraction of the full sub-mass range, in $[0, 1]$. Severity > 0.35 raises a console warning.

Each epoch produces a `WALD` telemetry line in `training.log`, parsed by the dashboard alongside `MYCELIUM` and `TLIGHT`:

```
WALD epoch=497 step=12600 fill=38.2% mass=8.641
    dead_low=6.25-7.75(1.50) dead_high
        =10.50+(1.25)
    severity=0.441 n=290 coverage
        =[1,3,8,45,234,891,...]
```

Relationship to Mycelium. Mycelium tracks *expert-space* health: which routing nodes are alive, which are crystallising, where gradient pressure concentrates. Wald tracks *loss-space* health: which regions of the error landscape the model has never visited. They are orthogonal views of the same training run. A healthy Mycelium report with a severe Wald dead zone indicates that the experts are well-distributed but the corpus lacks examples that push the model into lower-loss territory—a curriculum gap, not a routing problem.

Distinction from curiosity-driven RL. Curiosity-driven exploration (Schmidhuber [12], Pathak et al. [13]) seeks novelty in *state space*. The Wald monitor seeks coverage in *loss space*, and it is deliberately asymmetric: the objective is not general novelty but targeted reinforcement of the low dead zone specifically—the territory where the model's understanding is absent rather than merely untested. The future extension of this module is a curriculum bias signal: when `dead_low` persists across epochs, the corpus sampler oversamples inputs that have historically produced losses near that band, closing the Wald loop from passive observation into active curriculum steering.

H. Training Corpus Architecture

Albert's training pipeline follows a stage-aware curriculum, automatically expanded by the EvolutionManager as the model gains architectural depth:

TABLE VI
ALBERT V2.0.0 CORPUS CURRICULUM: DEPTH-GATED PEDAGOGICAL ARC

Stage	Unlock	Corpus	Size
3	3 layers	King James Bible + *Alice in Wonderland*	4.3 MB
6	6 layers	+ Simple English Wikipedia (factual grounding)	8.0 MB
7	7 layers	+ Linux documentation and man pages	4.3 MB
8	8 layers	+ 12 Gutenberg classics (Dickens, Austen, Tolstoy, et al.)	11.7 MB
9	9 layers	+ Instruction QA pairs	3.2 MB
10	10 layers	+ Human internet register (solved forum threads, recipes, trails)	5.8 MB

Total corpus at full architectural depth: **37.3 MB**, approximately 7.5 million tokens. The stage ordering follows a deliberate cognitive arc: structural syntax and literary anti-rigidity first (Bible + Alice), then factual encyclopedic grounding (Wikipedia), then technical precision (Linux docs), then long-range literary coherence (Gutenberg), then instruction following, then the informal register and ambiguity of real human communication (solved forum threads, recipes, trail guides). Each unlock happens automatically as the EvolutionManager fires surgery and increments the layer count, ensuring that representational challenges always outpace current architectural capacity.

I. Live Training Telemetry

A production-grade observability infrastructure accompanies the training loop:

- **HTTP Range Request tail-fetch**: only the final 50 KB of log data is downloaded per dashboard poll, eliminating the multi-minute buffering delay that plagues full-log streaming.

- **Every-batch logging with OS-level disk flushing**: `f.flush()` called after every batch write, reducing terminal-to-dashboard latency from $\sim$50 s to $<$3 s.
- **5000+ real-time data points** rendered without frame-drop via Chart.js index-aware state management, with logarithmic loss scaling for long-horizon comparison across evolutionary phases.
- **Global Odometer**: persists total training epoch mileage across sessions (Global Epoch 477+ at time of writing), providing a continuous measure of model maturity independent of session boundaries.
- **Routing entropy (`ENTR`)**: $-H(\mathbf{p})/\text{num\_layers}$ emitted per step, providing real-time diagnosis of expert specialisation health ($\ln 12 = 2.485$ at uniform; post-intervention target: 2.29–2.37).
- **Load-balancing telemetry**: per-step `LB val=X.XXXX` emission enables continuous monitoring of auxiliary loss magnitude and convergence of the routing distribution toward balanced expert utilisation.

J. Benchmark Results

Sustained Inference Throughput. Measured at Global Epoch 350 on the 4-layer configuration (HP ZBook i7-4800MQ, x86 CPU, 8 threads via Rayon, Ubuntu 24.04 LTS, no GPU, no INT8 kernel, no framework acceleration), the full inference stack achieves **83 tokens/second** sustained decode. Three multiplicative contributions:

1) **`@sparseskip` expert routing** [11]: 9/12 experts not executing per decode step (4.58$\times$ contribution at 75% sparsity; Patent A50296/2026).
2) **KV-cache**: single query token attends over cached K/V, eliminating recomputation of all prior context positions.
3) **Pre-ternarized weight cache**: `prepare_inference()` quantizes weights once at model load; all forward passes use the cached ternary representation with no per-step re-quantization.

Two independent sparsity layers. Albert delivers savings at two orthogonal levels: (1) **Routing-level sparsity** (`@sparseskip`): 9 of 12 experts are entirely skipped per decode step at the MoE gate—75% of expert computation is never executed. (2) **Weight-level sparsity** (ternary zeros): within each active expert tensor, 10–26% of individual weight trits are zero (TELE telemetry; L0 at 10.6%, rising monotonically to L11 at 26.5%). The 83 tok/s figure reflects routing-level savings only. Weight-level zero-elision via the `TSPARSE_MATMUL` primitive is a *distinct, additive* optimisation currently in development; applying it to the active expert tensors would multiply throughput further on both x86 and dedicated ternary hardware. On native trit ASICs (the SPRIND roadmap target), the theoretical figure from hardware simulation reaches 122$\times$ over conventional binary execution.

Current State: 17-Layer v3.0 (Global Epoch 795+). The EvolutionManager grew Albert from 12L to 17L across five autonomous surgery events (epochs 511, 547, 611, 645, 701). Since the fifth surgery, the model has continued training uninterrupted, descending steadily without triggering the plateau gate. Key live measurements at time of writing:

- **All-time best loss**: 10.2199 (epoch-best, Global Epoch 786; random baseline $\ln(32{,}000) \approx 10.37$). A cascade of 22+ consecutive new-best epochs from Epoch 743 to Epoch 786 established this record.
- **Descent rate**: ≈ 0.015 nats per 10 epochs, sustained over the most recent 50-epoch window.
- **MYCELIUM gradient hierarchy**: L3 has held the hot-layer position (highest gradient norm) for 89 consecutive epochs at Epoch 790, confirming deep layer-crystallisation. Early layers L0–L2 are nearly frozen ($\approx 1.9 \times 10^{-9}$ grad norm); L3 remains the active learning frontier.
- **WALD weight-distribution health**: severity 0.982, histogram fill 2.1% (halved from 4.2% at Epoch 781—the distribution is compressing toward the mean, a sign of healthy consolidation).
- **Routing entropy**: ENTR $\approx$ 2.47, slightly elevated but within the monitored range; load-balancing auxiliary loss active.
- **Expert health**: dead-expert resurrections occurring continuously; 5 experts blooming at Epoch 790. The MYCELIUM system distributes gradient signal to cold experts autonomously.

Language Modelling Perplexity. Formal perplexity evaluation was conducted at Global Epoch 107 on a held-out excerpt of *Alice's Adventures in Wonderland* (English), under the 32,000-token ByteLevel BPE tokenizer: cross-entropy 10.3668, yielding PPL = 31,788 versus the 32,000-token unigram baseline. The model had trained for only 107 epochs at this measurement; training has continued substantially since. At Global Epoch 786 (best loss 10.2199, $\Delta = 0.147$ nats below the 32k random baseline), the equivalent PPL is approximately 27,500—a 14% reduction versus the unigram ceiling and still actively descending. Full formal perplexity evaluation across all eight v3.0 languages is scheduled for Stage 2 milestones, at which point the model will be substantially deeper into its training curve.

Surgery Governor: Empirical Validation. The EvolutionManager's distinguishing property is that it withholds surgery when surgery is not warranted. At Global Epoch 791, all three preconditions for the sixth surgery were technically satisfiable—the Fibonacci cooldown had expired (post-surgery window of 89 epochs elapsed at Epoch 702+89=791), and the MYCELIUM stability gate was satisfied (myc_stable=89, well above the threshold of 5). The plateau gate withheld surgery because the 89-epoch loss window showed ≈ 0.07 nats of descent—well above the 0.02 threshold. Training continued at 17L. This non-event is as significant as any of the five prior surgeries: the governor distinguishes genuine plateau from active learning, and this distinction has now been demonstrated in both directions under live production conditions.

K. Albert v3.0: Multilingual European Foundation

The v2.0.0 chapter proves the full Albert stack works on CPU from random initialisation: 12 transformer layers grown autonomously, expert routing self-organised without intervention, zero expert deaths across hundreds of monitored epochs. The architecture is validated. The limitation that

remains is linguistic scope: the original 8,000-token English WordLevel vocabulary cannot represent German compound nouns, French morphological variants, or Slavic inflectional paradigms without catastrophic tokenisation inefficiency.

v3.0 closes this gap without discarding what v2.0.0 built. The 12-layer transformer blocks—attention weights, expert MLPs, MoE routing matrices, TTL state—are vocabulary-agnostic. They operate on hidden-state vectors, not token IDs. Only the embedding matrix (`embed.weight`) and language model head (`lm_head.weight`) are tied to vocabulary size. The v3.0 upgrade path is therefore:

1) Archive the v2.0.0 final checkpoint at Global Epoch 500.
2) Train a 32,000-token ByteLevel BPE tokenizer on a balanced European corpus (EN, DE, FR, ES, PT, IT, NL, PL), using the HuggingFace `tokenizers` library. ByteLevel BPE encodes any Unicode script without UNK tokens, exactly matching the approach used by GPT-2 and RoBERTa.
3) Transfer the 687 vocabulary-agnostic tensors (`blocks.0` through `blocks.11`, `ln_f.*`, `pos_embed.weight`) into the v3.0 init checkpoint.
4) Rebuild `embed.weight` and `lm_head.weight` at the new vocabulary size using Kaiming uniform initialisation (seed 42 for reproducibility).
5) Resume training on the multilingual European corpus, monitoring TTL routing heatmaps for language-correlated expert specialisation.

Training data provenance. The v3.0 corpus is drawn entirely from public domain and open-licence sources: Wikipedia article dumps (DE, FR, ES, PT, IT, NL, PL) under CC BY-SA 4.0; the Brockhaus Encyclopaedia 14th edition (1894–1896) via the Internet Archive (public domain); Europarl parliamentary proceedings (EU open data licence); Gutenberg multilingual literature (public domain); and EU regulatory texts including the AI Act in all official languages (EU open data). Layered above this clean base is an academic abstracts tier (HAL archives-ouvertes.fr, arXiv, Zenodo OAI-PMH) and a full-paper tier (ar5iv, PubMed Central, PERSEE) covering 12 disciplines across all eight languages. No proprietary data, no scraped web content, no personally identifiable information. This satisfies EU AI Act Article 10 training data governance requirements and positions Albert as a demonstrably European sovereign AI with traceable, auditable training provenance.

The Uncertainty Tithe. Approximately 10 % of the v3.0 training corpus is a dedicated *chaos layer*—28 MB of structured uncertainty distributed across seven types:

Type	Budget	Description
Redacted corpus	27%	Literary and technical text with [MASK] / [REDACTED] tokens at 10–40% density
Truncated reasoning	20%	Arguments cut at 25–75% of natural length, no resolution
Open questions	15%	Unanswered StackExchange posts (Math, CSTheory, Philosophy, Linguistics, Physics)
Masked academic	10%	Abstract text with 20–40% word-level masking
Failed trades	10%	Synthetic trade setups (60% losing) + SEC enforcement actions
OCR corruption	10%	Character-level noise: l↔1, o↔0, dropped glyphs
Code-switched	8%	English paragraphs with 1–2 injected foreign-language sentences

This is intentional. A model trained exclusively on clean, resolved text acquires a dangerous prior: it assumes the world closes. The chaos layer shifts that prior. Albert learns—at the statistical baseline—that roughly one text in ten has no clean answer, no resolution, no correct outcome. This *90/10 uncertainty tithe* is not a concession to data scarcity. It is a design choice. The ratio is enforced programmatically: `train_tokenizer_v3.py` measures the chaos fraction at tokenizer training time and emits a build warning if it falls outside the 8–14% window.

The tithe is expressed at four independent levels of the stack: the routing entropy floor (ENTR ≥ 2.480) prevents the output distribution from collapsing to certainty; the Ternary Traffic Light defaults to Orange (unresolved) rather than Green (committed); `@sparseskip` withholds 75 % of expert capacity on every decode step; and the chaos corpus ensures that training exposure itself is never pristine. Together these form a single architectural commitment—**epistemic humility as a design primitive**, not an alignment patch applied after the fact.

v3.0 Training Status (May 2026). At the time of this submission, albert. v3.0 has completed over 795 global epochs at 17 layers, with an all-time best loss of 10.2199 ($\Delta = 0.147$ nats below the 32,000-token random baseline of $\ln(32{,}000) \approx 10.37$). Five autonomous surgeries have expanded the model from 12L to 17L without manual intervention or architecture search. The sixth surgery gate is armed but has not fired—the model is actively descending and the plateau condition is not yet met. Stage 10 of the curriculum is active, covering the full eight-language corpus. Training continues on Modal GPU (NVIDIA T4) with real-time telemetry streamed to a live dashboard at each epoch boundary. The architecture, training loop, and growth governor are production-grade; the current run is not a proof-of-concept but a live, self-governing research organism accumulating linguistic competence epoch by epoch.

XII. EU AI Act Compliance Architecture

The EU AI Act establishes risk-tiered obligations for AI systems deployed in Europe. High-risk systems must satisfy

Articles 13 (transparency), 14 (human oversight), and 15 (accuracy, robustness, and cybersecurity). The Ternary Intelligence Stack is the first AI execution substrate where compliance with these three articles is *structurally native* rather than bolt-on.

A. The Natural Regulatory Correspondence

The three trit values map directly to three regulatory stances:

TABLE VII
TERNARY TRIT STATES AND EU AI ACT REGULATORY CORRESPONDENCE

Trit	State	Article	Regulatory response
$+1$	Affirm	Art. 13	Transparency satisfied: output may be acted upon
0	Hold	Art. 14	Human oversight required: system withholds decision
-1	Conflict	Art. 15	Safety measure triggered: action blocked, audit logged

This correspondence is not incidental. The EU AI Act's requirement that high-risk systems express uncertainty rather than hallucinate confident incorrect answers is precisely what the trit hold state encodes at the language and ISA level. Binary systems must retrofit uncertainty quantification on top of deterministic outputs; ternary systems produce it natively.

B. ternaudit-guard

The `ternaudit-guard` crate provides the operational compliance layer:

- **Decision log processing**: Parses JSON-RPC audit logs emitted by the BET VM and maps each decision record to the corresponding Article 13–15 requirement.
- **Regulatory mapping layer**: Converts internal confidence metrics (Affirm / Tend / Reject) to structured compliance reports with per-decision regulatory status.
- **Immutable audit trail**: Every safety veto and hold decision is logged to the Axis-tier memory with timestamp, expert identity, and query hash—satisfying the Article 15 robustness and cybersecurity logging requirements.
- **VS Code extension integration**: Inline verification markers in TernStudio IDE allow developers to audit AI-generated code with per-logic-segment compliance status before deployment.

C. The Tend State as Alignment Primitive

Binary AI systems face a structural alignment problem that compliance frameworks cannot resolve: every output must be a decision. A system that does not know must still answer. The result is hallucinated confidence—commitment to the best available answer even when the correct answer is *insufficient information*.

The ternary 0 state resolves this at the representation level. When a ternary reasoning chain encounters contradictory evidence, model uncertainty, or a query whose resolution requires information not yet in scope, the output is not a forced binary choice but a **tend**—a probabilistic lean accompanied by an explicit acknowledgment that the conclusion is conditional and subject to revision. This is not abstention; it is the most informationally accurate statement the system can make given current evidence.

> *"I currently tend toward this conclusion—but routing entropy is high. Additional context would allow me to commit."*

This property is implemented at multiple levels of the TIS stack:

- In the **BET VM**: division-by-zero or unresolvable trit propagation produces 0 (tend), not a crash or `NaN`. The computational analogue of honest uncertainty is a native output type, not an exception.
- In the **MoE-13 Orchestrator**: the hold condition ($|\text{affirm} - \text{conflict}| \leq 1$ across ≥ 4 agents) blocks commitment to a majority-vote result rather than forcing one, returning a hold trit.
- In **Albert's routing**: before the load-balancing intervention resolved expert homogeneity, the model's uniform routing was itself a tend—the gate could not distinguish experts and correctly produced a diffuse distribution. The LB loss did not suppress this signal; it created the expert differentiation necessary to resolve it.
- In the **EU AI Act correspondence** (Table VII): trit 0 maps to Article 14 human oversight—the system signals that this decision requires human review rather than automated action.

The alignment significance of this property is architectural, not policy-layer. An AI system that natively outputs *tend* is a categorically different epistemic actor than one constrained to return binary yes/no. Hallucination—the production of confident incorrect output—is not a training failure of binary systems alone; it is a consequence of forcing three-valued epistemic states (know, don't know, know-but-uncertain) into a two-valued output type. The ternary tend state is not a workaround for this problem. It is a first-class output of a system designed from first principles to represent uncertainty as real information.

D. GDPR and Data Sovereignty

All TIS components execute on-premise or within EU jurisdiction (Fly.io Frankfurt). No training data, model weights, or inference results transit to non-EU infrastructure. The open-core licensing (LGPL-3.0-or-later for the compiler and CLI; BSL-1.1 converting to LGPL in 2030 for the ML engine) ensures that the sovereignty guarantee is contractually enforceable, not merely a runtime policy.

XIII. QUTRIT NEURAL NETWORKS AND TERNLANG

Terminology note. QNN here denotes *Qutrit Neural Networks*—classical simulations of three-state $(-1, 0, +1)$ qutrit-valued weight matrices executing on conventional silicon—and does not require, depend on, or claim quantum entanglement, superposition hardware, or quantum gate circuits. The correspondence to quantum qutrit formalism is mathematical and structural, not physical. No quantum hardware is assumed or required at any stage.

The Qutrit Neural Network (QNN) framework [14] establishes a formal correspondence between quantum qutrit states and balanced ternary values: $|-\rangle \leftrightarrow -1$, $|0\rangle \leftrightarrow 0$, $|+\rangle \leftrightarrow +1$.

This correspondence is not coincidental. Balanced ternary's symmetric structure around zero and its active neutral state mirror the physical properties of three-state quantum systems, making ternlang a natural implementation substrate for QNN inference. The `@sparseskip` directive applies directly to qutrit weight matrices: at realistic qutrit sparsity levels ($\approx 33\%$ zero states at initialisation), the `TSPARSE_MATMUL` kernel provides comparable speedup to classical BitNet workloads. The repository contains 15 QNN-inspired `.tern` programs covering qutrit superposition, ternary attention mechanisms, TRS stabilisation loops, and qutrit gradient approximation.

XIV. Related Work

Balanced ternary computing. Knuth [4] provides the mathematical foundation. The Setun computer (1958) [15] demonstrated physical ternary hardware. Modern revivals include academic hardware studies at the University of South-Eastern Norway [6], targeting C-to-ternary compilation for EDA tools—an approach that inherits binary-native semantics and misses the symmetric properties that TIS exploits natively.

Ternary emulators. Several open-source projects implement ternary VMs in Python or JavaScript without compiler, ML kernel, or hardware backend. `ternlang-compat` provides assembler-level bridges to the most significant of these.

Ternary quantization for neural networks. BitNet [5] and BitNet b1.58 [16] demonstrate that large language models can be trained with ternary weights while retaining competitive perplexity. The present work is the first to: (a) surface this property as a first-class ISA opcode; (b) train a language model natively within a ternary execution stack rather than applying quantization post-hoc; and (c) provide hardware-verified profiling of the four distinct microarchitectural regimes that govern real-world sparsity speedup.

Sovereign AI infrastructure. The EU's AI Act and the broader digital sovereignty agenda have driven interest in auditable, on-premise AI execution. TIS is the first full-stack architecture where regulatory compliance is structurally native to the execution model rather than implemented as middleware.

XV. The Ternary Computing Ecosystem

A stated goal of TIS is to serve as a *convergence point* for the fragmented ternary computing field. Table VIII maps active projects to their interoperability path via `ternlang-compat`.

Beyond runtime interoperability, TIS serves as a *technical archive* for prior ternary computing work. Brandon Smith's Python 9-trit RISC simulator demonstrated a complete fetch-decode-execute cycle in balanced ternary—an existence proof that received little downstream adoption due to the absence of a compiler ecosystem. Both Smith's pipeline and the Owlet interpreter are honoured in the `examples/` corpus as runnable `.tern` programs on the same VM that drives the MoE-13 orchestrator.

TABLE VIII
Ecosystem compatibility map

Project	Interface	Bridge
Brandon Smith 9-trit sim	`.tasm` assembly	`TasmAssembler` → BET bytecode
Owlet (S-expr)	S-expression	`OwletParser` → ternlang AST
USN / EDA tools	C-to-ternary	Academic whitepaper; ISA interop
BitNet b1.58	Ternary weights	`moe-llm-core` weight ingest
Q-Ternary	Qutrits	`trittensor` state model (future)

XVI. Implementation Status

The Ternary Intelligence Stack is implemented in Rust and comprises **34 published crates** at ecosystem version **v1.3.7** (`crates.io`). Table IX lists the primary crates; the full manifest is maintained in the repository.

Technology Readiness Assessment

On the hardware driver gap. The audit trail correctly identifies that bare-metal drivers for physical ternary silicon are absent from the repository. This is not a gap in the research programme—it is the *definition* of what SPRIND Stage 2 and Stage 3 funding is designed to produce. The BetRtlProcessor RTL simulator (TRL 7) is the verifiable hardware blueprint: every logic gate, register, and instruction cycle is specified in synthesisable Verilog-2001. The path from TRL 2 to TRL 6 for silicon drivers requires access to a ternary ASIC fabrication partner and the cluster compute to validate the full training stack at scale—both contingent on the funding this application seeks. Evaluators should read the RTL simulator as the engineering commitment, not the absence of drivers as a programme weakness.

CI-verified test suite: 138+ fast tests across five self-contained crates (core: 43, codegen: 8, moe: 24, compat: 29, hdl: 34); network-bound crates validated locally. **Live deployment**: REST API and MCP endpoint at `https://ternlang.com/mcp` (Fly.io Frankfurt, TLS via Let's Encrypt). **IDE tooling**: VS Code extension (`ternlang-0.1.0`) published on Open VSX Registry (publisher `rfi-irfos`), providing syntax highlighting, LSP diagnostics, and snippet completion. **Example corpus**: 275+ executable `.tern` programs spanning aerospace, medicine, distributed systems, hardware pipelines, civic governance, finance, AI agent design, and ten Qutrit Neural Network modules. **CI/CD**: GitHub Actions automatically builds, tests, and deploys to Fly.io on every push to `main`.

XVII. Conclusion

We have presented the **Ternary Intelligence Stack v1.3.7**—a sovereign, full-stack post-binary architecture for neural intelligence, built by RFI-IRFOS from Graz, Austria. Eleven dimensions of contribution are unified under the foundational primitive of the trit $t \in \{-1, 0, +1\}$ with an active neutral state:

1) **Language and execution substrate**: Ternlang with exhaustive three-way matching, TSPARSE_MATMUL achieving up to $2.84\times$ wall-clock speedup (hardware-verified, four-regime model), and a native C11 compilation path.
2) **Hardware**: Synthesisable Verilog-2001 BET processor with per-cell clock gating and an Icarus Verilog simulation wrapper.
3) **Reasoning infrastructure**: Five-tool Ternary AI Reasoning Toolkit (deliberation, coalition vote, action gate, temperature bridge, hallucination score) and the MoE-13 v2.0 orchestrator with 13-agent dual-signal deliberation and an introspective hold stable attractor.
4) **Auto-Evolutionary Training**: Albert-MoE-13 with EvolutionManager plateau-mastery state machine; Net2Net Safe Copy Surgery with Mandelbrot-parameterised fractal symmetry break; Mycelium expert health monitor with autonomous resurrection; Wald loss-space coverage monitor; TTL routing with anti-stagnation burst; 90/10 uncertainty tithe; nine autonomous surgeries from 3L to 12L; sub-3-second terminal-to-dashboard telemetry latency.
5) **Regulatory compliance**: ternaudit-guard providing the first language-native EU AI Act mapping where trit $+1$/0/-1 correspond structurally to Articles 13, 14, and 15.
6) **Sovereignty**: 34 published crates, all under open-core LGPL licensing, deployed exclusively within EU jurisdiction, with no dependency on proprietary AI infrastructure.
7) **Ecosystem**: 275+ example programs, VS Code LSP extension, MCP server, REST API, `ternpkg` package manager, and interoperability bridges to the broader ternary computing field.

The ternary computing field has been fragmented for decades. TIS is designed to be the substrate where those fragments converge—from ISA to orchestrator, from single trit to 13-expert MoE, from native EU compliance to quantum-adjacent qutrit networks. Europe does not need to catch up. It needs to leap ahead.

Binary sees yes and no. Ternary also sees *not yet*—and that changes everything.

Near-term targets: submission to VS Code Marketplace; FPGA synthesis of the full `bet_processor` on Xilinx Artix-7 and Lattice ECP5; Albert-MoE-13 sub-2.0 loss convergence on the consolidated knowledge corpus; Hindley-Milner type inference to eliminate explicit trit type annotations; formal academic outreach to the USN memristor hardware group for joint hardware-software co-design; Phase 22 commercialisation targeting industrial sovereign AI deployments in regulated sectors (healthcare, legal, defense-adjacent, critical infrastructure).

REFERENCES

[1] S. Kepp, Z. Karimi, N. Csonka, L. Scharler, and L. P. Ehrig, "Ternary intelligence stack — full-stack sovereign ai infrastructure," https://github.com/eriirfos-eng/ternary-intelligence-stack, 2026, open-source. RFI-IRFOS, Graz. Patent Pending A50296/2026.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022. [Online]. Available: https://jmlr.org/papers/v23/21-0998.html

[3] S. Kepp and Z. Karimi, "Albert-moe-13 benchmark harness (`moe-test`)," https://github.com/eriirfos-eng/ternary-intelligence-stack/tree/main/albert-moe-13/moe-test, 2026, provides `--bench` mode: inference throughput, `@sparseskip` analysis, and perplexity evaluation. Reproducing: `cargo run --release -p moe-test -- --bench`.

[4] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, 1997, section 4.1: Positional Number Systems; balanced ternary pp. 190–192.

[5] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei, "The era of 1-bit LLMs: All large language models are in 1.58 bits," *arXiv preprint arXiv:2402.17764*, 2024. [Online]. Available: https://arxiv.org/abs/2402.17764

[6] S. Bos and H. Gundersen, "Ternary logic synthesis for CMOS technology using electronic design automation," in *Proceedings of the Norwegian Informatics Conference*, 2020, university of South-Eastern Norway.

[7] S. Kepp and Z. Karimi, "SPRIND Scientific Artifact: Sparsity performance validation for TSPARSE_MATMUL — hardware-verified profiling of microarchitectural bottleneck transitions," RFI-IRFOS Technical Report, 2026, patent Pending A50296/2026. Bootstrap resampling $n = 1{,}000$; $\Delta\text{AIC} = 1683.29$; breakpoint at 10.06% sparsity, 95% CI $[10.00\%, 10.38\%]$.

[8] S. Kepp, "MoE-13: Ternary mixture-of-experts orchestration architecture," OSF Preprints, 2026, rFI-IRFOS. Dual-key synergistic routing, 1+1=3 triad synthesis, three-tier memory mesh, safety hard gate.

[9] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, "BitNet: Scaling 1-bit transformers for large language models," *arXiv preprint arXiv:2310.11453*, 2023.

[10] S. Kepp and Z. Karimi, "Albert-moe-13 training orchestrator (`train_bible.rs`)," https://github.com/eriirfos-eng/ternary-intelligence-stack/blob/main/albert-moe-13/moe-llm-core/src/bin/train_bible.rs, 2026, implements Net2Net surgery, Fibonacci EvolutionManager, Mandelbrot symmetry break, WALD loss-space coverage, TTL routing, and stage-aware corpus curriculum.

[11] S. Kepp, "`@sparseskip`: Zero-overhead sparse execution for ternary neural network inference," Patent Pending A50296/2026, 2026, rFI-IRFOS. TSPARSE_MATMUL opcode; expert-level MoE routing skip; $4.58\times$ speedup at 75% expert sparsity on x86 CPU. Reproducing: `cargo run --release --bin sparseskip_throughput`.

[12] J. Schmidhuber, "A possibility for implementing curiosity and boredom in model-building neural controllers," *Proceedings of the International Conference on Simulation of Adaptive Behavior*, pp. 222–227, 1991.

[13] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 2778–2787.

[14] S. Kepp, "Qutrit neural networks: Balanced ternary foundations for quantum-adjacent inference," https://osf.io/, 2026, rFI-IRFOS. Qutrit states $|-\rangle/|0\rangle/|+\rangle$ mapped to trit $\{-1, 0, +1\}$; Tesseract Recursive Syntax (TRS) stabilisation.

[15] N. P. Brousentsov, S. P. Maslov, J. Ramil Alvarez, and E. A. Zhogolev, "Development of ternary computers at Moscow State University," *Russian Virtual Computer Museum*, 2002. [Online]. Available: http://www.computer-museum.ru/english/setun.htm

[16] S. Ma, H. Wang, L. Ma *et al.*, "BitNet b1.58: Large language models in the ternary regime," *arXiv preprint*, 2024.

TABLE IX
TIS CRATE MANIFEST AND DESCRIPTIONS (V1.3.7)

Crate	Role
`ternlang-core`	Lexer, parser, AST, semantic checker, BET VM (51 opcodes)
`ternlang-ml`	BitNet quantization, sparse/CSC matmul, reasoning toolkit
`ternlang-moe`	MoE-13 v2.0 orchestrator: dual-key routing, triad synthesis
`ternlang-codegen`	AST-to-C11 transpiler
`ternlang-test`	Full-pipeline test framework: `TernTestCase`, `assert_tern!`
`ternlang-hdl`	Verilog-2001 codegen, BET RTL simulator
`ternlang-runtime`	Distributed TCP actor runtime (`TernNode`)
`ternlang-compat`	`.tasm` assembler (9-trit RISC), Owlet S-expr parser
`ternlang-lsp`	LSP 3.17: hover, completion (19 snippets), diagnostics
`ternlang-mcp`	MCP server, 34 tools: `moe_orchestrate`, `trit_action_gate`, …
`ternlang-api`	REST API (18 endpoints), multi-tenant key management, SSE
`ternlang-compress`	Ternary compression primitives
`ternlang-ruvector`	SIMD-aligned ternary vector operations
`ternpkg`	Package manager, GitHub-backed registry
`ternlang-⋆`	14 additional domain crates: `-hdl`, `-wasm`, `-sec`, `-hft`, `-ros2`, `-qutrit`, `-sql`, `-ui`, `-web`, `-translator`, `-harmony`, `-swarm`, `-consensus`, `-crypto`
`albert-cli`	Sovereign intelligence CLI (model-agnostic)
`albert-runtime`	Albert training and inference runtime
`albert-api`	Albert REST API
`albert-commands`	CLI command registry
`albert-tools / compat`	Tooling and compatibility bridge
`moe-core`	Internal MoE-13 inference engine
`moe-platform`	Stable API for MoE-13
`moe-plugin-sdk`	SDK for third-party expert plugins
`moe-runtime`	Training/inference runtime
`moe-llm-core`	TritTransformer architecture
`moe-compute`	Kernel acceleration and compute graph
`moe-validation-suite`	Reproducibility and benchmark harness
`moe-uril / moe-ddel / moe-sdk / moe-test`	Utility, deployment, and test layers
`pytern`	Python bindings
`ternaudit-guard`	EU AI Act compliance, JSON-RPC audit
`moe-reference`	Reference implementation (v1.3.6)
`moe-llb`	Low-level backend (v1.3.6)

TABLE X
TRL ASSESSMENT BY SUBSYSTEM (2026-05-10)

Subsystem	TRL	Basis
Ternlang compiler + BET VM	7	Operational prototype
BetRtlProcessor RTL simulator	7	Cycle-accurate Verilog
Sparse inference (`gemv_bitnet`)	6	122× speedup measur
MCP server (34 tools, live API)	7	Production deploymen
Albert-MoE-13 native training	4	Validated on single C
`@sparseskip` routing (Patent A50296/2026)	5	Measured 75% expert
Distributed actor runtime	4	Implemented; large-sc
Native ternary silicon drivers	2	Design modelled in B